Module 1 - INTRO DSA

Module 1

Introduction to DSA

Look around and you will see that people organize things:

- Write TO-DO Lists
- Queue at ATMs
- Arrange stacks of books
- Look-up words in dictionary
- Organize directories in computers
- Plan road map to go somewhere

Introduction

Data Structure can be defined as the group of data elements which provides an efficient way of storing and organizing data in the computer so that it can be used efficiently.

- Some examples are arrays, Linked List, Stack, Queue, etc.
- Widely used in almost every aspect of Computer Science.
    - i.e., Operating System, Compiler Design, Artifical intelligence, Graphics and many more.

Terminologies

- **Data**
    - elementary value or the collection of values. Such as student's name or ID.
- **Group Items**
    - data items with subordinate data items. Such as student's name consisting of first and last name.
- **Record**

    - collection of various data items. Such as the record for student which contains the student's name, address, contact details and etc.
- **File**
    - Collection of various records. If there are 40 students in DSA, there are 40 records pertaining to those students.
- **Attribute and Entity**
    - Entity represents the class, attribute represents property.
- **Field**
    - A single elementary unit of information representing the attribute of an entity.

Why do we need Data Structures?

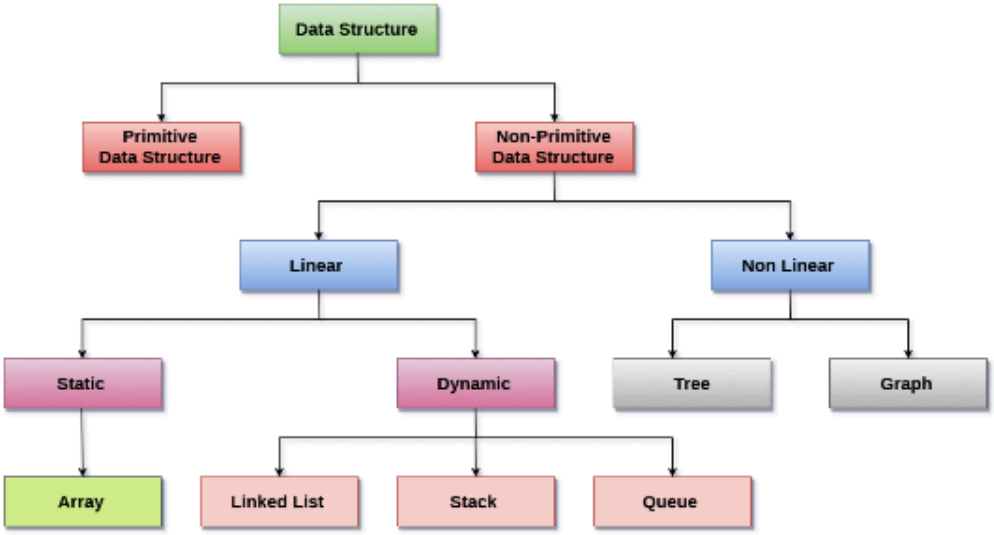
As our applications become more complex, with the more and more data used, issues may arise considering:

- Processor Speed
- Data Search
- Multiple Requests

Advantages of Data Structures

- The **efficiency** of a program depends on what data structure is being used.
- Data structures are **reusable**.
- Data structure specified by the abstract data type provides a level of **abstraction**.

Classification



Linear Data Structures

A data structure is called linear if all of its elements are arranged in the linear order.

- In linear data structures, the elements are stored in non-hierarchical way where each element has the successors and predecessors except the first and last element.
- Array:
    - A collection of similar type of data items, with each data item referred to as an element.
- Linked List:
    - A collection of nodes stored at non-contiguous memory locations.
- Stack:
    - A linear list in which insertion and deletions are allowed only at one end, the top.

- Queue:
    - A linear list in which elements can be inserted only at one end called rear and deleted only at the other end called front

Non-Linear Data Structures

This data structure does not form a sequence.

- i.e., each item or element is connected with two or more other items in a non- linear arrangement.
- The data elements are not arranged in sequential structure.
- Trees:
    - Multilevel data structures with a hierarchical relationship among its elements that are called nodes.
- Graphs:
    - The pictorial representation of the set of elements (represented by vertices) connected by the links known as edges.

Operations on Data Structures

- Traversing
- Insertion
- Deletion
- Searching
- Sorting
- Merging

Pointers

- A pointer is a variable that holds the memory address of another variable of the same type.

  int *ip; // pointer to an integer

  double *dp; // pointer to a double

  float *fp; // pointer to a float

  char *ch // pointer to character

- This memory address is the location of another variable where it has been stored in the memory.
- It supports dynamic memory allocation routines.

```cpp
#include <iostream>
using namespace std;

int main (){
    int var = 20; // actual variable declaration.
    int *ip; // pointer variable

    ip = &var; // store address of var in pointer variable

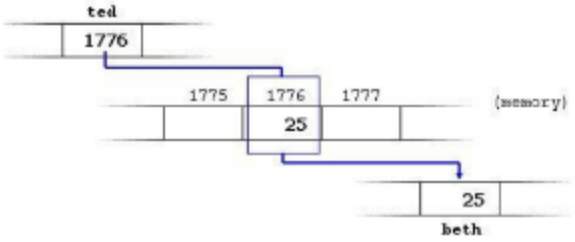
    cout << "Value of var variable: ";
    cout << var << endl;

    // print the address stored in ip pointer variable
    cout << "Address stored in ip variable: ";
    cout << ip << endl;

    // access the value at the address available in pointer
    cout << "Value of *ip variable: ";
    cout << *ip << endl;

    return 0;
}
```

```
beth = *ted;
```



int bar = *foo_ptr;
*foo_ptr = 42;

& is the reference operator and can be read as "address of"
* is the dereference operator and can be read as "value pointed by"

Allocating Memory

There are two ways that memory gets allocated for data storage:

1. Compile Time (or static) Allocation
    - Memory for named variables is allocated by the compiler
    - Exact size and type of storage must be known at compile time
    - For standard array declarations, this is why the size has to be constant
2. Dynamic Memory Allocation
    - Memory allocated "on the fly" during run time
    - dynamically allocated space usually placed in a program segment known as the heap or the free store
    - Exact amount of space or number of items does not have to be known by the compiler in advance.
    - For dynamic memory allocation, pointers are crucial

Dynamic Memory Allocation

- We can dynamically allocate storage space while the program is running, but we cannot create new variable names "on the fly"
- For this reason, dynamic allocation requires two steps:
    1. Creating the dynamic space.
    2. Storing its address in a pointer (so that the space can be accesed)
- To dynamically allocate memory in C++, we use the new operator.
- De-allocation:
    - Deallocation is the "clean-up" of space being used for variables or other data storage
    - Compile time variables are automatically deallocated based on their known extent

      (this is the same as scope for "automatic" variables)
    - It is the programmer's job to deallocate dynamically created space
    - To de-allocate dynamic memory, we use the delete operator

Allocating space with new

- To allocate space dynamically, use the unary operator new, followed by the type being allocated.

```
new int; // dynamically allocates an int
new double; // dynamically allocates a double
```

- If creating an array dynamically, use the same form, but put brackets with a size after the type:

```
new int[40]; // dynamically allocates an array of 40 ints
new double[size]; // dynamically allocates an array of size doubles
```

```
// note that the size can be a variable
```

- These statements above are not very useful by themselves, because the allocated spaces have no names! BUT, the new operator returns the starting address of the allocated space, and this address can be stored in a pointer:

```
int * p; // declare a pointer p
p = new int; // dynamically allocate an int and load address into p
double * d; // declare a pointer d
d = new double; // dynamically allocate a double and load address into d


// we can also do these in single line statements
int x = 40;
int * list = new int[x];
float * numbers = new float[x+10];
```

- Notice that this is one more way of initializing a pointer to a valid target (and the most important one).

Accessing dynamically created space

- So once the space has been dynamically allocated, how do we use it?
- For single items, we go through the pointer. Dereference the pointer to reach the dynamically created target:

```
int * p = new int; // dynamic integer, pointed to by p
*p = 10; // assigns 10 to the dynamic integer
cout << *p; // prints 10
```

- For dynamically created arrays, you can use either pointer-offset notation, or treat the pointer as the array name and use the standard bracket notation:

```
double * numList = new double[size]; // dynamic array
```

```
for (int i = 0; i < size; i++)
numList[i] = 0;// initialize array elements to 0
numList[5] = 20; // bracket notation
*(numList + 7) = 15; // pointer-offset notation
                                // means same as numList[7]
```

Deallocation of dynamic memory

- To deallocate memory that was created with new, we use the unary operator delete. The one operand should be a pointer that stores the address of the space to be deallocated:

```
int * ptr = new int; // dynamically created int
// …
delete ptr;  // deletes the space that ptr points
```

- Note that the pointer ptr still exists in this example. That's a named variable subject to scope and extent determined at compile time. It can be reused:

```
ptr = new int[10]; // point p to a brand new array
```

- To deallocate a dynamic array, use this form:

```
delete [] name_of_pointer;
```

- Example:

```
int * list = new int[40]; // dynamic array


delete [] list; // deallocates the array
list = 0; // reset list to null pointer
```

- After deallocating space, it's always a good idea to reset the pointer to null unless you are pointing it at another valid target right away.

Memory Leak

- A memory leak occurs when a piece (or pieces) of memory that was previously allocated by a programmer is not properly deallocated.
- Even though the memory is not used by the program anymore, it is still “reserved” and that piece of memory cannot be used by the program until it is properly deallocated.
- It is called a memory leak because it works like a leaky faucet where water is wasted – only in this case, it is the computer memory.

Review of Arrays

- Arrays are defined as the collection of similar type of data items stored at contiguous memory locations.
- Arrays are the derived data type in C programming language which can store the primitive type of data such as int, char, double, float, etc.
- Array is the simplest data structure where each data element can be randomly accessed by using its index number.

Properties of the Array

- Each element is of same data type and carries a same size i.e. int = 4 bytes.
- Elements of the array are stored at contiguous memory locations where the first element is stored at the smallest memory location.
- Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of data element.

Need of using arrays

```cpp
#include <iostream>
using namespace std;

int main ()  {
    int marks_1 = 56, marks_2 = 78, marks_3 = 88,
    marks_4 = 76, marks_5 = 56, marks_6 = 89;
    float avg = (marks_1 + marks_2 + marks_3 + marks_4 + marks_5 +marks_6) / 6 ;
    cout << avg;

    return 0;
}
```

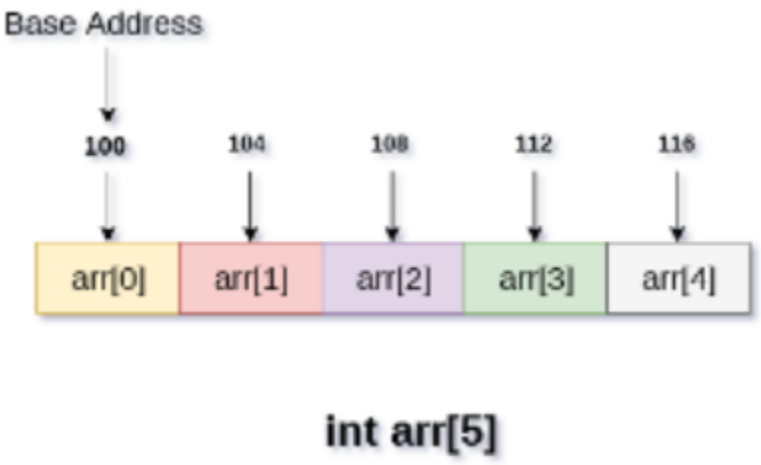
```cpp
#include<iostream>
using namespace std;

int main(){
    int marks[6]={56,78,88,76,56,89};
    int i;
    float avg = 0;
    for(i = 0; i<6; i++){
        avg = avg + marks[i];
    }
    cout << avg/6;

    return 0;
}
```

Memory Allocation of the array

- Indexing is defined in three ways:
    - 0 based indexing
    - 1 based indexing
    - N based indexing



n the following image, we have shown the memory allocation of an array arr of size 5. The array follows 0-based indexing approach. The base address of the array is 100th byte. This will be the

address of arr[0]. Here, the size of int is 4 bytes therefore each element will take 4 bytes in the memory.

Abstract Data Types (ADTs)

- Data abstraction, or abstract data types, is a programming methodology where one defines not only the data structure to be used, but the processes to manipulate the structure
    - like process abstraction, ADTs can be supported directly by programming languages
- To support it, there needs to be mechanisms for
    - defining data structures
    - encapsulation of data structures and their routines to manipulate the structures into one unit
        - by placing all definitions in one unit, it can be compiled at one time
    - information hiding to protect the data structure from outside interference or manipulation
        - the data structure should only be accessible from code encapsulated with it so that the structure is hidden and protected from the outside
        - objects are one way to implement ADTs, but because objects have additional properties, we defer discussion of them until the next chapter

ADT Design Issues

- Encapsulation:

  it must be possible to define a unit that contains a data structure and the subprograms that access (manipulate) it

    - design issues:
        - will ADT access be restricted through pointers?
        - can ADTs be parameterized (size and/or type of data being stored)?
- Information hiding: controlling access to the data structure through some form of interface so that it cannot be directly manipulated by external code
    - often implemented via two sections of code
        - public part (interface) constitutes those elements that can be accessed externally (often limited to subprograms and constants)
        - private part, which remains secure because it is only accessible by subprograms of the ADT itself

ADTs in C++

- C++ offers two mechanisms for building data structures: the struct and the class
    - because the struct does not have a mechanism for information hiding, it can only offer encapsulation (and encapsulation is not enforced when using structs, merely available), so for a true ADT, we must use C++s object
    - C++ classes contain both visible (public) and hidden (private) components (as well as protected for inheritance)
    - C++ instances can be static, heap-dynamic and stack-dynamic
        - the lifetime of an instance ends when it reaches the end of the scope of where it was declared
        - a stack-dynamic object may have heap-dynamic data so that parts of the object may continue even though the instant is deallocated
    - we defer most of our discussion of objects in C++ to the next chapter, but we will see an example next

Parameterized ADTs

- The ability to define an ADT where the type and/or size is specified generically so that a specific version can be generated later
    - a stack defined without specifying the element type (integer vs. string vs. float, etc)
    - a stack defined without a restriction on the size of the stack
    - Ada, C++, Java and C# all have this capability
- The approach is to replace the type definition with a place holder that is filled in later
- In C++, parameterized ADTs are implemented as templated classes
    - to change the stack class' size, only change the constructor to receive the size as a parameter, which is used to establish the size of the array
    - to change the stack's type, the class is now defined as a template using template <class Type> where Type is the place-holder to be filled in by a specific type at runtime
- In both Ada and C++, the parameterized ADT definition is generated at compile-time
    - the new statement signals that a new package should be generated by the compiler
        - in C++, if two definitions ask for the same type of ADT, only 1 set of source code is generated, in Ada, the same source code is generated twice!
- In Java and C#, parameterized ADTs are implemented as generic classes.

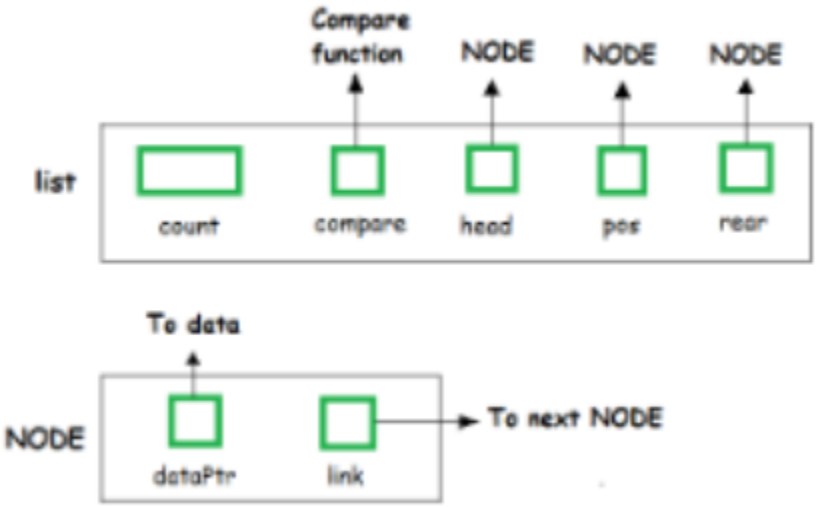
Common ADTs

- Consider ADT as the black box.
- 3 common ADTs:

- List ADT
- Stack ADT
- Queue ADT

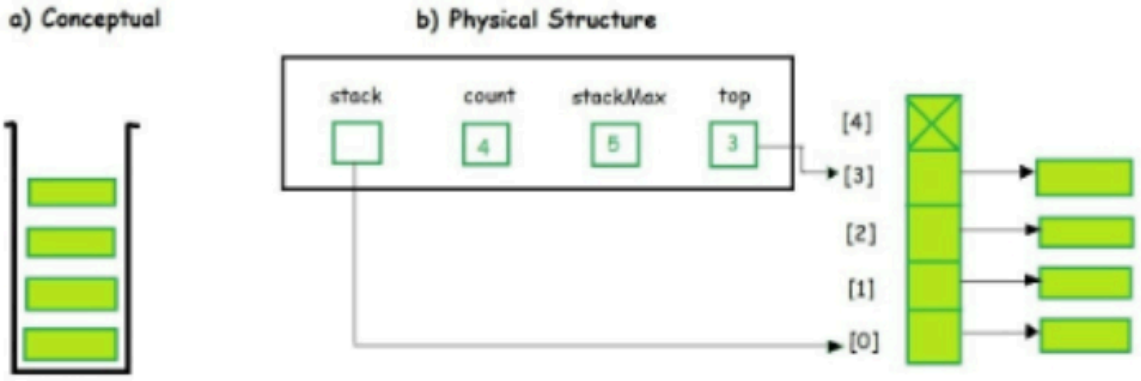
List ADT

- The data is generally stored in key sequence in a list which has a head structure consisting of count, pointers and address of compare function needed to compare the data in the list.



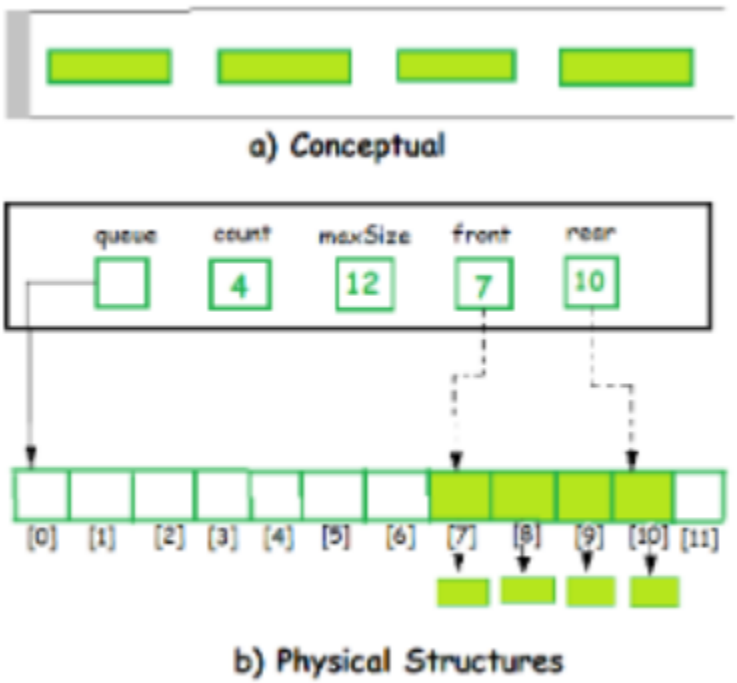
Stack ADT

- In Stack ADT Implementation instead of data being stored in each node, the pointer to data is stored.
- The program allocates memory for the data and address is passed to the stack ADT.



Queue ADT

- The queue abstract data type (ADT) follows the basic design of the stack abstract data type.

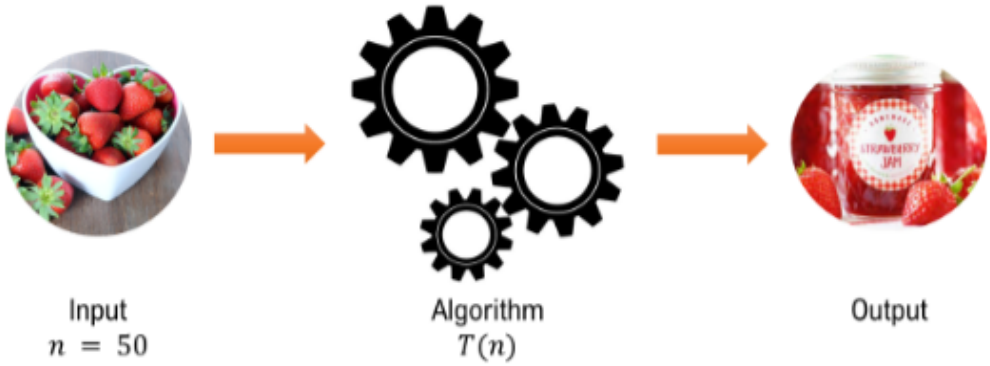


Module 1 - Algo and Algo Analysis

Module 1 - Algo and Algo Analysis

Introduction to Algorithms and Algorithm Analysis

Based on the slides of Young CS 331 D&A of Algo. Topic: Introduction w/ content from Analysis of Algorithms cs.purdue.edu



What is an **algorithm**?

A simple, unambiguous, mechanical procedure to carry out some task.

Why algorithm instead of program?

1. Writing an algorithm is simpler (we don't need to worry about the detailed implementation, or the language syntax).
2. An algorithm is easier to read than a program written in, for instance, C.

How to represent an algorithm?

1. Give a description in your own language, e.g. English, Spanish, …
2. Pseudo code
3. Graphical

Example – multiplying two positive integers A and B

For example: 45*19

Usually:

```
  45
  19   (x
-----
 405
  45   (+
-----
 855
```

A different algorithm:

Multiplier (A/2)	Multiplicand (B*2)	Result (pick numbers in column 2 when the corresponding number under the multiplier is odd)
45	19	19
22	38	
11	76	76
5	152	152
2	304	
1	608	608 (+
		855

An instance of a problem is a specific assignment of values to the parameters.

This algorithm can be used for multiplying any two positive integers, we say that (45, 19) is an **instance** of this problem. Most problems have infinite collection of instances.

It's ok to define the **domain** (i.e. the set of instances) to be considered, and the algorithm should work for all instances in that domain.

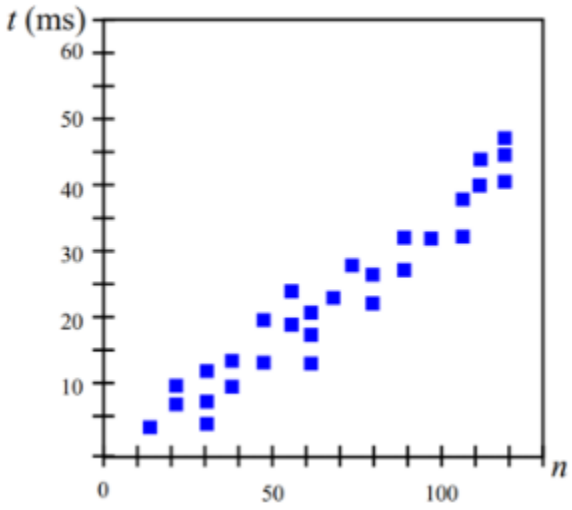
Although the above algorithm will not work if the first operand is negative, this does not invalidate the algorithm since (-45, 19) is not an instance of the problem being considered.

How to measure the running time of an algorithm?

- Experimental Study
- Theoretical Analysis

In experimental study, write the program that implements the algorithm. Then run the program with datasets of varying size and composition. Use a method like #include<chrono> methods to get accurate measure of actual running time.

Show the resulting data like shown below.



Experimental studies have limitations:

- Algorithm must be implemented to test its running time.
- Experiments can only be performed on limited set of inputs.
- Comparison will be dependent on computing environment.

Instead, we also use general methodology of theoretical analysis:

- Use high-level description over implementation.
- Consider all possible inputs.
- Evaluate efficiency that is agnostic to computing environment.

Order:

Usually we use the frequency count to compare algorithms.

Consider the following 3 programs:

(a)

```
x←x+y
```

(b)

```
for i ← 1 to n do
        x←x+y
end
```

(c)

```
for i ← 1 to n do
        for j ← 1 to n do
                x←x+y
        end
end
```

The frequency count of stmt x ← x + y is 1, n, n2.

No matter which machine we use to run these programs, we know

that the execution time of (b) is n times the execution time of (a).

Pseudo-code is a description of an algorithm that is more structured than usual prose but less formal than a programming language.

Example: Find the maximum element of an array

```
Algorithm arrayMax(A, n):
    Input: An array A storing n integers.
    Output: The maximum element in A.
  currentMax ← A[0]
  for i ← 1 to n −1 do
    if currentMax < A[i] then
      currentMax ← A[i]
  return currentMax
```

Pseudo-code is our preferred notation for describing algorithms. However, pseudo-code hides program design issues.

What are the asymptotic notations?

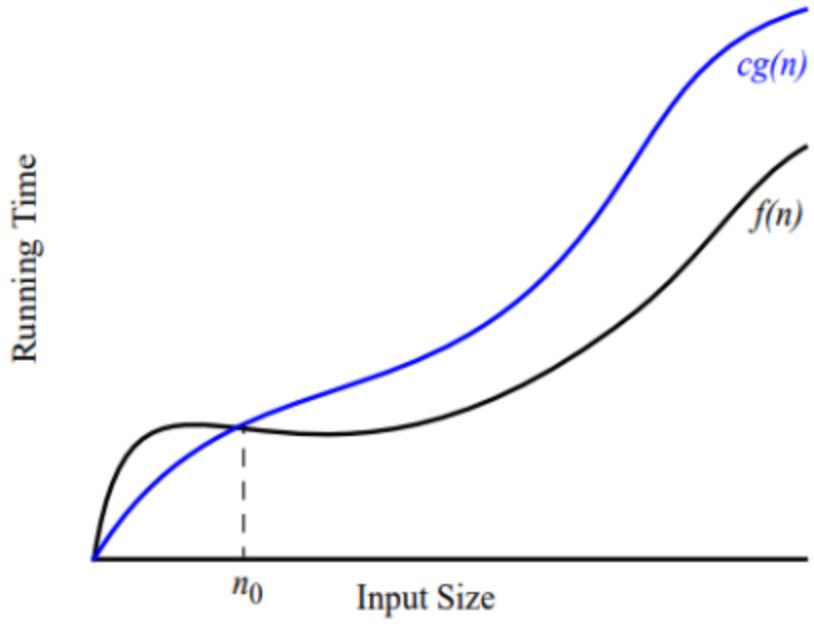
These asymptotic notations are used to describe the running time of a given algorithm when the input approaches a particular or limiting value. These are:

- Big Oh Notation (O)
  - $T(N) = O(f(n))$ if there are no positive constants c and n_0 such that $T(N) \leq cf(N)$ when $N \geq n_0$.
- Omega Notation (Ω)
  - $T(N) = \Omega(g(N))$ if there are positive constants c and n_0 such that $T(N) \geq cg(N)$ when $N \geq n_0$.
- Theta Notation (Θ)
  - $T(N) = \theta(h(N))$ if and only if $T(N) = O(h(N))$ and $T(N) = \Omega(h(N))$.
- Little Oh Notation (o)
  - $T(N) = o(p(N))$ if for all positive constants c, there exists an n_0 such that $T(N) < cp(N)$ when $N > n_0$.

Keep this example in mind… We know that 1000*N* > N^2 when the value of N is small. N^2 grows at a faster rate and will eventually be the bigger function. After N = 1000, N^2 will start to overtake 1000*N*.

The “Big-Oh” Notation

Given functions f(n) and g(n), we say that: f(n) is O(g(n)) if and only if f(n) ≤ c g(n) for n ≥ n_0



The goal for asymptotic notation is to simplify the analysis by removing the unnecessary information.

$$1{,}000{,}000 \approx 1{,}000{,}003$$

$$3n^2 = n^2$$

$f(n) = 4n^3 + 3n^2$ can be approximated as $O(n^5)$ or even $O(n^4)$, it is actually $O(n^3)$

Examples:

- Is $17\ n^2 - 5 = O(n^2)$?

$$\therefore \qquad 17n^2 - 5 = O(n^2)$$

- Is $35\ n^3 + 100 = O(n^3)$?

$$\therefore \qquad 35n^3 + 100 = O(n^3)$$

- Is $6 \bullet 2^n + n^2 = O(2^n)$?

$$\therefore \qquad 6 \cdot 2^n + n^2 = O(2^n)$$

Complexity classes

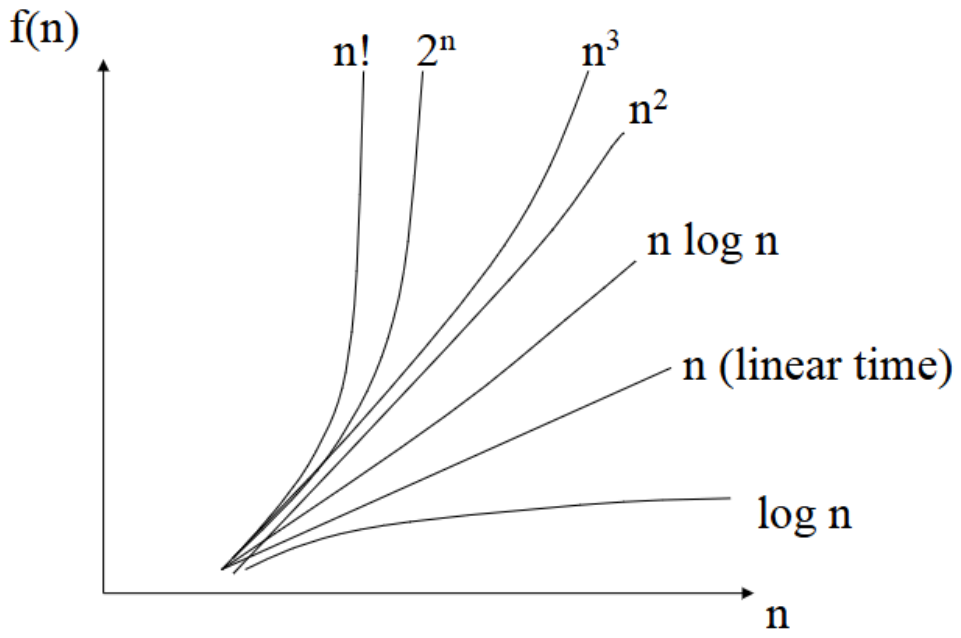


Figure 2. Growth rates of some important complexity classes

Simple rules:

- If $f(n)$ is a polynomial of degree d, then $f(n)$ is $O(n^d)$. We can drop the lower order terms and constant factors.
- Use the simplest expression of the class, example: $3n + 5$ is O(n) not O(3n)

Assume that we have a machine that can execute 1,000,000 required operations per sec

	Algorithm 1	**Algorithm 2**	**Algorithm 3**	**Algorithm 4**	**Algorithm 5**
Frequency count	33n	6nlogn	$13n^2$	$3.4n^3$	2^n
n=10 **n=10,000**	< 1 sec < 1 sec	< 1 sec < 1 sec	< 1 sec 22 min	< 1 sec 39 days	< 1 sec many many centuries

Table 1. Execution time for algorithms with the given time complexities

Note:

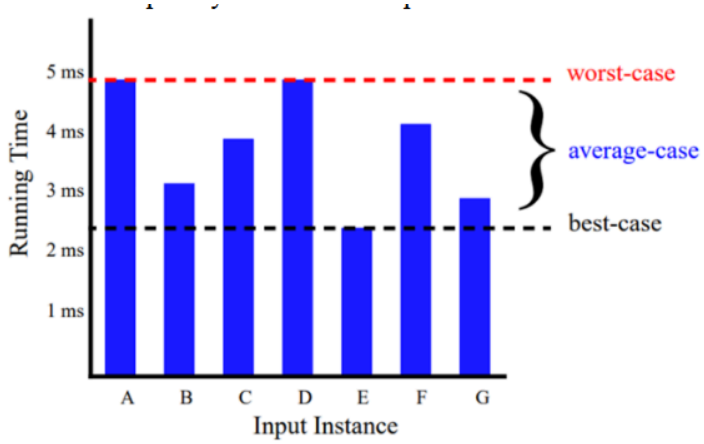
log n
n(linear)
n log n
n^2
n^3
} polynomial time (easy or tractable)

2^n
n!
} exponential time (hard or intractable)

Analysis of Algorithms

a) Best Case, Worse Case Analysis

An algorithm may run faster on certain data sets than on others, Finding the average case can be very difficult, so typically algorithms are measured by the worst-case time complexity. Also, in certain application domains (e.g., air traffic control, surgery) knowing the worst-case time complexity is of crucial importance



Module 2 - Arrays Pointers and DMA

Module 2 - Arrays Pointers and DMA

Pointers, Dynamic Memory Allocation, and Built-in Arrays

CPE010 – Data Structures and Algorithms

Intended Learning Outcomes:

After this module, the student should be able to:

a. Implement static and dynamic memory allocation
b. Create dynamically allocated objects using pointers and arrays
c. Solve programming problems using dynamic memory allocation, arrays and pointers

"Normal" Variable declarations

Normal variable instantiation automatically assigned a free memory address to a variable and stores that value in that memory address.

```
int x = 10;
```

x represents an alias for the memory address that holds the value 10.

When compiled, the variable name gets translated to it's appropriate memory address.

Reference Operator (&)

The reference operator (&) is used to retrieve the memory address assigned to a variable

```
int x = 10;
cout << x; // prints value 10
cout << &x; // prints the memory address of x
```

We use the reference operator when using function / method parameter passing by reference.

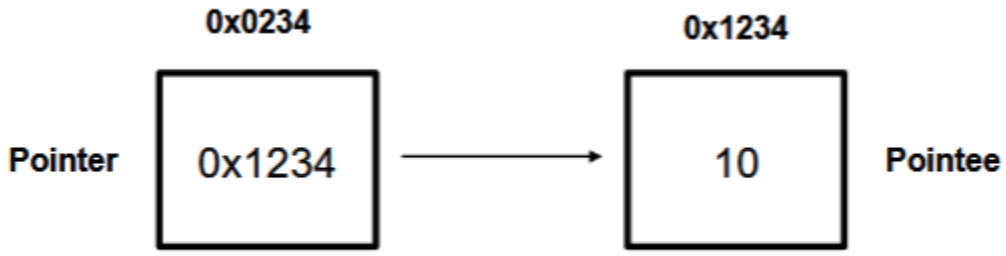
Dereference Operator (*)

The dereference operator (*) allows you to access the value at a particular memory address.

```
int x =  10;
cout <<  x; // prints value 10
cout <<  &x; // prints the memory address of x
cout <<  *&x; // prints the value 10
```

Pointers

A Pointer is a variable that contains the memory address of another location in memory that contains information.



Pointer Declaration

To declare a pointer, you need to specify the variable type of the pointee and provide the asterisk between the data type and the name.

```
int *iPtr;
double *dPtr;
```

It is best practice to declare pointers on their own lines.

It is also best practice to define the function return type as a pointer with the asterisks next to the data type, not the function name.

Pointer Assignment

Remember that pointers only hold a memory address. When we assign a value to a pointer, the value must be a memory address.

Draw the memory diagram for the following:

```
int value = 5;
int *ptr = &value;
```

nullptr

nullptr

- null pointer keyword is used to indicate a pointer points to nothing.

A pointer assigned to nullptr is said to be null.

0x0234

Pointer 0x0

example: nullptr.cpp

Dynamic Memory

In this class, up until this point, we have only been concerned with data / memory that has been allocated on the stack.

With Dynamic Memory Allocation memory is allocated during runtime.

Dynamic Memory allocation uses the Heap.

C++ Memory

- 4 Main Memory Parts Memory
  - Code
  - Static (Global) Variables / Data Stack
  - Stack
  - Heap
- Heap and Stack vary dynamically Heap
- Code and Static/Global use is fixed

Static (Global)

- Code is read-only

Code

The new keyword

new: used for allocating memory (dynamic memory allocation)

The new operator allocates memory during runtime and is persisted independently of any particular function.

When we use the new keyword. The memory for that data is allocated on the heap.

Memory Diagram Exercise

Draw the memory diagram for the following:

1. int *ptr = nullptr;

2. int valu = 10;
   int e = &value;
   *ptr

3. int *ptr = new int;

4. int *ptr = new int(10);

Implement Swap with pointers

Lets implement swap using pointers. Here is how we implement swap with normal variables:

```
void swap(int &x, int &y) {
  int temp = x;
  x = y;
  y = temp;
}
```

Arrays

Arrays - Built in arrays in C++ are built using pointers!

```
int array[] = {1, 2, 3, 4};
```

The above actually creates a pointer called array that we can use to access elements inside the array.

The compiler knows that the [ ] syntax indicates an array of integer values.

Arrays

One of the only differences in arrays declared with [ ] or using pointers is that we can reassign pointers and not arrays.

Arrays are always declared const when using [ ] notation.

```
int array[] = {1, 2,          3, 4};
int * ptrArray = {1,          2, 3, 4};
array = ptrArray; //          compiler error
ptrArray = array; //          ok
```

Dynamically Allocated Arrays

We can also dynamically allocate arrays

```
int *array = new int[10];
```

We can access arrays on the heap, the same way as before:

```
array[index];
```

However initialization is different:

```
int *array = {1,2,3,4}; // error
```

The delete operator

The delete operator does the opposite of the new operator.

This tells the OS to de-allocate or free up the memory block that is pointed to by the

```
delete ptr;
```

A general rule of thumb is for every new operator, there must be a delete operation.

delete operator more…

Memory Leak - a memory leak occurs when a programmer forgets to de-allocate memory that is pointed to by a pointer, then deletes the pointer or re-assigns the pointer to a new value.

Common Mistakes:

-allocating memory in a loop and calling delete on the pointer outside of the loop.

-not deleting data within an object that is dynamically allocated

delete[ ] operator

When dynamically allocating array's, we use the delete[ ]

operator to indicate that we are freeing a block of memory.

```
int *array = new int[5];

delete[] array;
```

Pointers with Objects/Classes

It is very common that we use pointers and the new keyword to create objects.

```
Student *a = new Student;
```

When the program executes this line, we first allocate the space for Student on the heap, then we call the Student default constructor.

We can also use other overloaded constructers for a class during initialization:

```
Student *a = new Student;
Student *a = new Student("Erik");
```

Member Access Operator

Accessing data members of a class can be done like the following:

```
Student *a = new Student;
(*a).data_member;
(*a).function();
```

However, with C++ and classes, we can use the member access operator (->) as an alternative (and easier) way of accessing data members or functions of a class.

```
a->data_member;
a->function();
```

Destructors

A destructor is a member function that is called which destructs or deletes an object.

```
Employee::~Employee()
{
   /* Clean up data */
}
```

Destructors are called when:

1. A function ends
2. The program ends
3. A block containing local variables end
4. A delete operator is called

A class only has one destructor

Copy Constructor

A copy constructor is used for a class to make a copy of an existing instance.

We use constants when we want to customize or tell the program how to copy an object when we pass by value.

*Remember: Pass by value creates a copy of the data and performs any operations on that copy in the function.

Copy Constructors are implicitly defined by the compiler if you do not define one and performs a member-wise copy of the source object.

Copy Constructor

It is important that we declare a copy constructor by passing in the class we want to copy as const.

```
MyClass::MyClass(const MyClass &copyClass) {
  ...
}
```

The most common use for the copy constructor is when the class has raw pointers as member variables and we need to make a deep copy of the data.

Deep vs Shallow copy

Deep Copy - a complete full copy of all data members of a class, including dynamically allocated memory.

Shallow Copy - a member-wise copy where we only copy the data members and NOT any dynamically allocated memory.

Your computer performs a shallow copy by default if no copy constructor is provided.

Copy Assignment Operator

Often times we want to copy one object to another using the assignment operator.

The program will implicitly create a copy assignment operator for you and do a member-wise copy.

But again we want to do a deep copy with an objects dynamically allocated data.

```
MyClass& Myclass::operator=(const MyClass& copy) {

}
```

Rule of Three

If you have to define one of the following as a programmer,

then it is a good practice to define all of them.

- Destructor

- Copy Constructor

- Copy Assignment Operator

These are often known as the “big three”

Module 3 - Linked Lists

Module 3 - Linked Lists

Linked Lists

C++ Standard Template Library (STL)

The C++ STL (Standard Template Library) is a powerful set of C++ template classes to provide general-purpose classes and functions with templates that implement many popular and commonly used algorithms and data structures like vectors, lists, queues, and stacks.

At the core of the C++ Standard Template Library are following three well-structured components −

No.	Components
1	Containers are used to manage collections of objects of a certain kind. There are several different types of containers like deque, list, vector, map etc.
2	Algorithms act on containers. They provide the means by which you will perform initialization, sorting, searching, and transforming of the contents of containers.
3	Iterators are used to step through the elements of collections of objects. These collections may be containers or subsets of containers.

https://en.cppreference.com/w/

Recap: Struct vs Class in C++

- How are structures not secure?

Members of a class are private by default and members of a struct are public by default.

```cpp
#include<iostream>
using namespace std;

class TestA {
    int x;
};

struct TestB {
    int x;
};

int main(){
    TestA t1;
    t1.x = 20;

    TestB t2;
    t2.x = 20;

    return 0;
}
```

[Error] 'int TestA::x' is private within this context

When deriving a struct from a class/struct, default access-specifier for a base class/struct is public. And when deriving a class, default access specifier is private.

```cpp
#include<iostream>
using namespace std;

class Base{
    public:
        int x;
};

class DerivedFromBase : Base {
};

int main(){
    DerivedFromBase d;
    d.x = 20;

    return 0;
}
```

```cpp
#include<iostream>
using namespace std;

class Base{
    public:
        int x;
};

struct DerivedFromBase : Base {
};

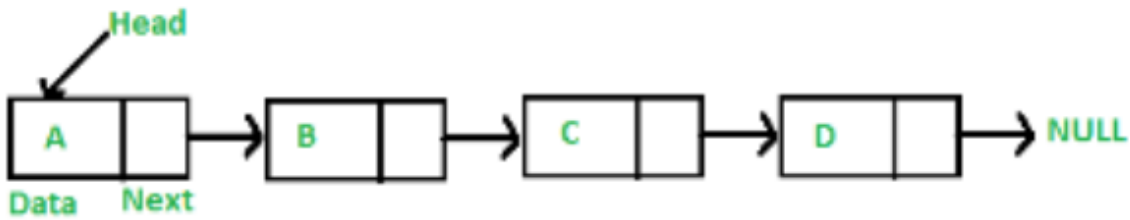
int main(){
    DerivedFromBase d;
    d.x = 20;

    return 0;
}
```

Inheritance: Private vs Public

Introduction to Linked Lists

Linked Lists are a linear data structure. Unlike arrays, linked list elements are not stored at a contiguous location; the elements are linked using pointers.



Why Linked List?

- Arrays are useful but have the following limitations:
    - Fixed size.
    - Allocated memory remains to be the upper limit.
    - Expensive operations (such as insertion) which requires movement of all existing elements and creation of room for new elements.

Linked Lists vs. Arrays

Advantages over Arrays

1. Dynamic size
2. Ease of insertion/deletion

Drawbacks

1. Random access is not allowed. We have to access elements sequentially starting from the first node
2. Extra memory space for a pointer is required with each element of the list.
3. Not cache friendly. No locality reference.

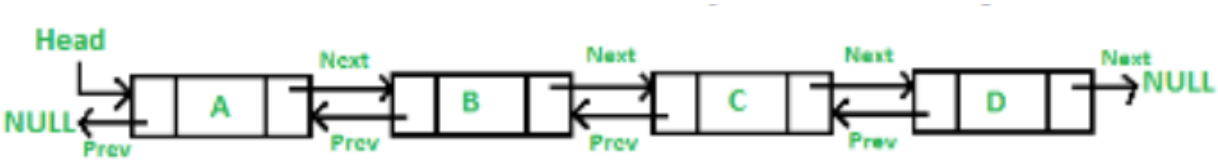
Representation:

- • A linked list is represented by a pointer to the first node of the linked list. The first node is called the head. If the linked list is empty, then the value of the head is NULL.
- Each node in a list consists of at least two parts:

1. Data
2. Pointer (Or Reference) to the next node

Doubly Linked Lists

Traversed in either direction. It is a linked list in which every node has a next pointer and a backpointer.



Every node contains address of next node (except the last node).

Every node contains address of previous node (except the first node).

Typical operations:

- Initialize the list
- Destroy the list
- Determine if list empty
- Search list for a given item
- Insert an item
- Delete an item, and so on

Simply Linked List

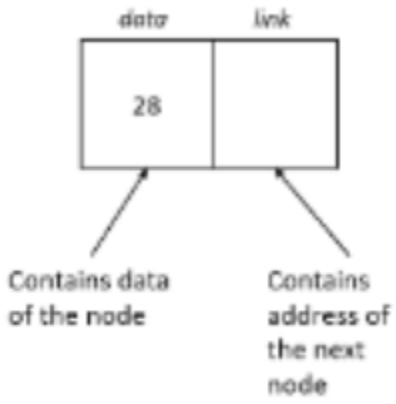
Create a simple linked list with your name, in this case "ROMAN"

Step 1: Create the node pointers

Step 2: Create new nodes and allocated in the heap

Step 3: Define every node in the list

Step 4: Point last node to NULL




```
class Node{
    public:
        char data;
        Node *next;
};
```

Simply Linked List: Implementation

```
#include<bits/stdc++.h>
using namespace std;

class Node{
    public:
        char data;
        Node *next;
};

//Create a simple linked list with your name, in this case "ROMAN"
int main(){
    Node *head = NULL;
    Node *second = NULL;
    Node *third = NULL;
    Node *fourth = NULL;
    Node *fifth = NULL;

    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
    fifth = new Node;

    //define first node
    head->data = 'R';
    head->next = second;

    //define second node
    second->data = 'O';
    second->next = third;

    //define third node
    third->data = 'M';
    third->next = fourth;

    //define fourth node
    fourth->data = 'A';
    fourth->next = fifth;

    //define fifth node
    fifth->data = 'N';
    fifth->next = NULL;

    return 0;
}
```

Linked List Traversal

```
Algorithm: ListTraversal (parameter: pointer to node n)
WHILE n IS NOT EQUAL TO null
        PRINT data OF n
        GO TO NEXT NODE n := next
        ENDWHILE
PRINT next line
END
```

Node Insertion

Sometimes we have a linked list and we need to insert a node somewhere other than at the end of the list. We will take a look at a couple of different ways to insert a node into an existing list

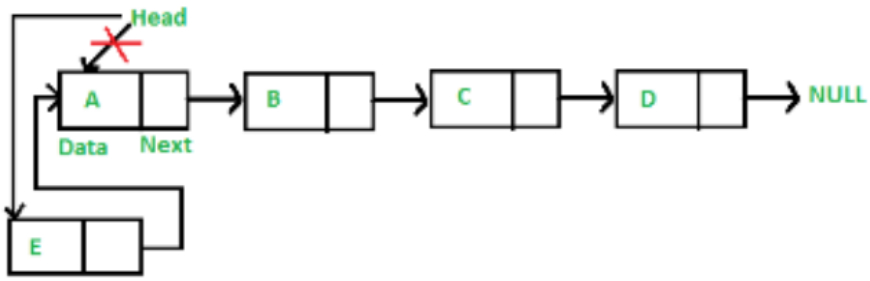
A new node can be added to a linked list in one of three ways

1. At the front of the linked list
2. After a given node.
3. At the end of the linked list.

Node Insertion at head

Steps:

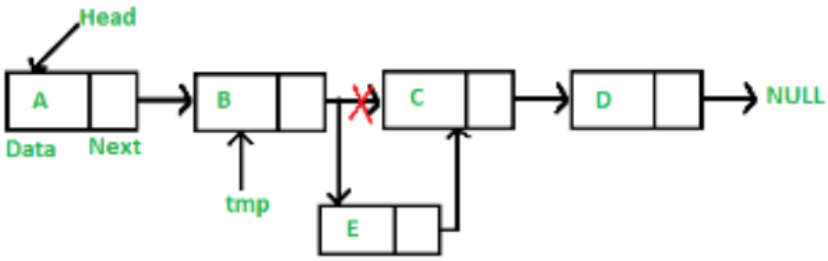
1. Allocate memory for the new node
2. Put our data into the new node
3. Set Next of the new node to point to the previous Head
4. Reset Head to point to the new node



General Node Insertion

Steps:

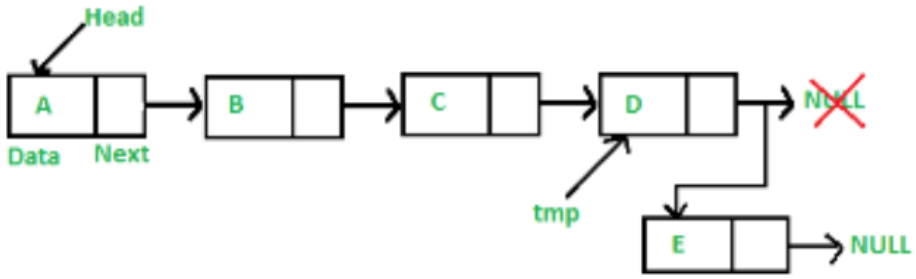
1. Check if it is the head node (previous node is null)
2. If null, print "Previous node cannot be null."
3. Allocate a new node
4. Store data in the new node
5. Point new node to the node previous node was pointing to
6. Point previous node to the new node



Insert Node at the End

Steps:

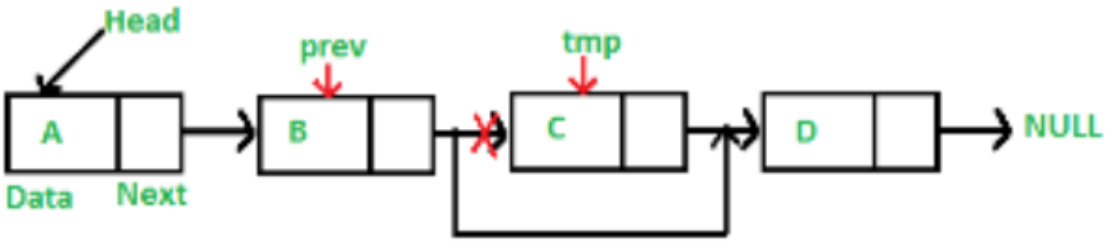
1. Allocate new node
2. Dereference to the head node
3. Store data in new node
4. Point next of new node to NULL
5. Traverse the list until next of the node is null
6. Point the next of the current node to the new node



Delete a Node

To delete a node from linked list, we need to do following steps.

1. Find previous node of the node to be deleted.
2. Change the next of previous node.
3. Free memory for the node to be deleted.



Module 4 - Stack

Module 4 - Stack

The Stack ADT

Stack at logical level

- A stack is an ADT in which elements add added and removed from only one end (i.e.,at the top of the stack).
- A stack is a LIFO “last in, first out” structure.

Stack Operations

Transformers (change state)

- Push
- Pop

Observers (observe state)

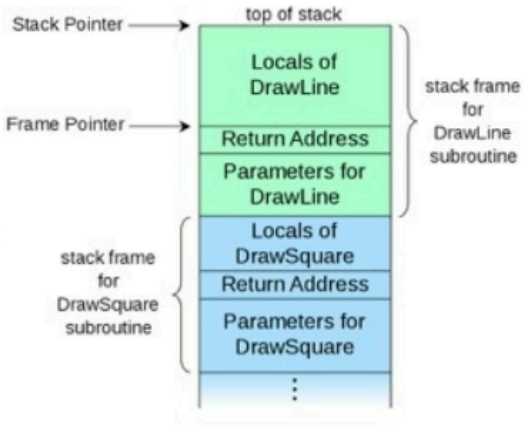
- Top
- IsEmpty
- IsFull

Stack at Application Level

- For what types of problems would be stack be useful for?
- LIFO: good for reversing data
    - If we push a, b, c, d into a stack, and then pop all elements out, we get d, c, b, a
- In OS/language: function call stack
- Finding Palindromes
- Expression evaluation and Syntax Parsing

Function call stack

```
void DrawSquare(int x, int y, int edge)
{
   DrawLine(x, y, edge, HORIZONTAL);
   DrawLine(x, y, edge, VERTICAL);
   DrawLine(x+edge, y, edge, VERTICAL);
   DrawLine(x, y+edge, edge, HORIZONTAL);
}
int main ()
{
   DrawSquare(1,2,3);
   return 0;
}
```



Use stack to reverse

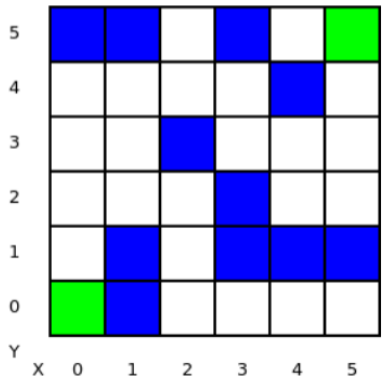
- Sometimes you need to output in reverse orders

  Convert decimal to binary: DisplayInBinary (int num)

  ```
  while (num>0)
      digit = num % 2
      print digit
      num = num / 2
  ```

- The binary representation is printed backward
  - Trace it with num=21, output?
- Solution: push each digit onto a stack in the loop, after the loop, pop digits out one by one and display it.
- In maze-walking algorithm, we can use stack to store all nodes that we led us to current node, so that we can backtrack when needed.

- Goal: find a path via white blocks from (0,0) to (5,5)
- Need to explore: try diff. next step
- and backtrack (when reach dead-end): i.e., reverse back to previous

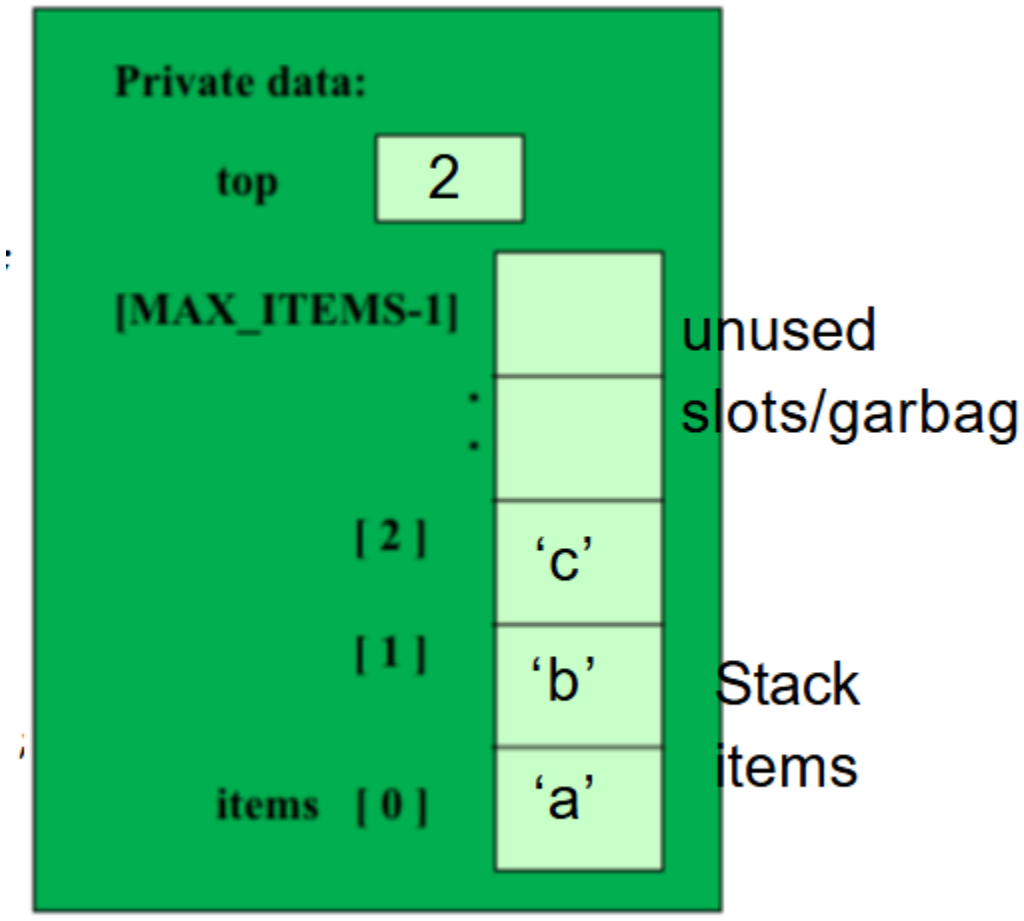


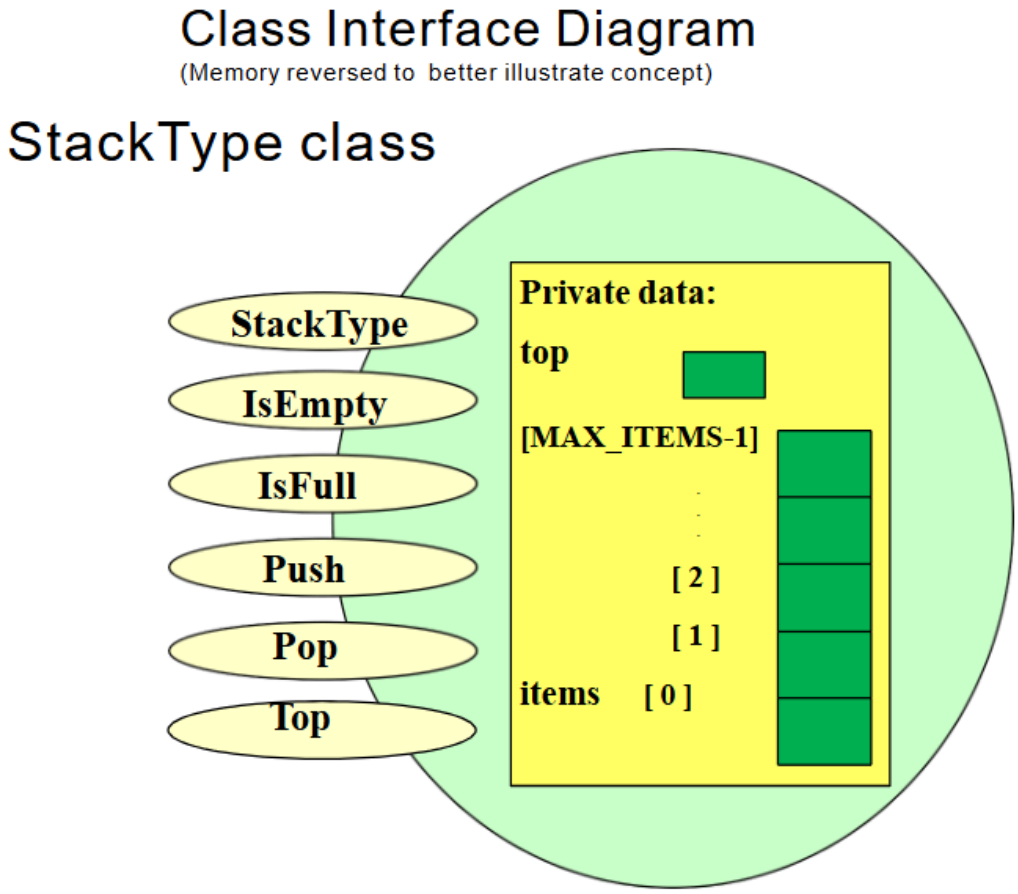
Stack Implementation

- array-based implementation: static or dynamic array
- linked-structure implementation

```
class StackType
{
public:
        StackType( );
        bool IsFull () const;
        bool IsEmpty() const;
        void Push( ItemType item );
        void Pop();
        ItemType Top();
private:
        int top;
```

```
    items[MAX_ITEMS];
    ItemType
};
```





Initialize stack

How to initialize data member top?

0 or -1

Depends on what top stores:

* initialized to 0: If it's the next open slot

*initialized to -1: if it's the index of stack top element

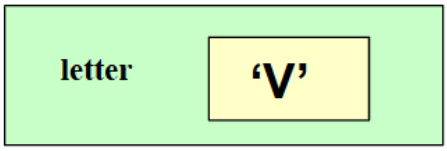
Need to be consistent in all member functions, Top(), Push(), Pop(), isfull, isEmpty()…

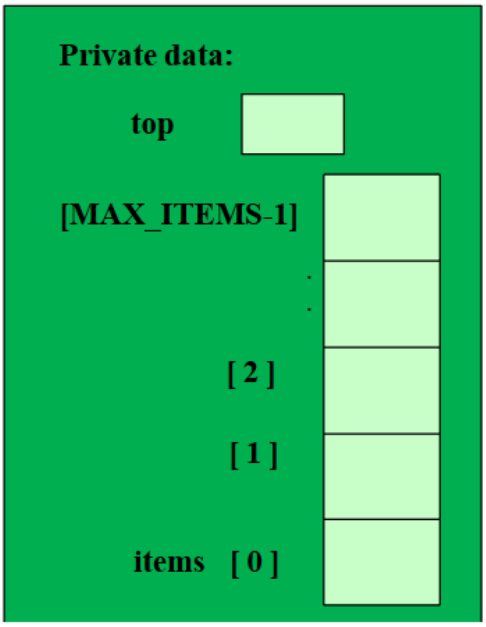
Below we use second option:

```
StackType::StackType( )
{
        top = -1; //index of top element in stack
}
// pre: the stack has been initialized
// post: return true if the stack is empty, false ow
bool StackType::IsEmpty() const
{
        return(top == -1);
}
//pre:
//post:
bool StackType::IsFull() const
{
        return (top = = MAX_ITEMS-1);
}
void StackType::Push(ItemType newItem)
{
        if( IsFull() )
                throw FullStack():
        top++;
        items[top] = newItem;
}
void StackType::Pop()
{
```

```
        if( IsEmpty() )
                throw EmptyStack();
        top--;
}
ItemType StackType::Top()
{
        if (IsEmpty())
                throw EmptyStack();
        return items[top];
}
```

Tracing Client Code

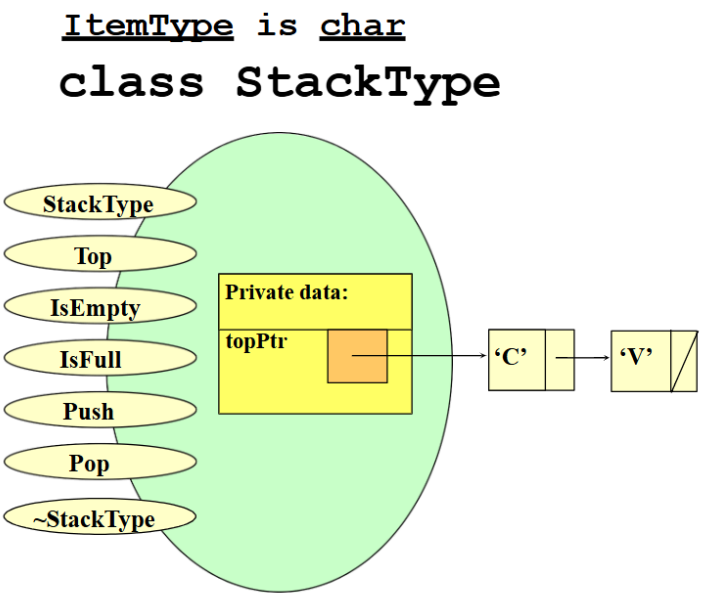




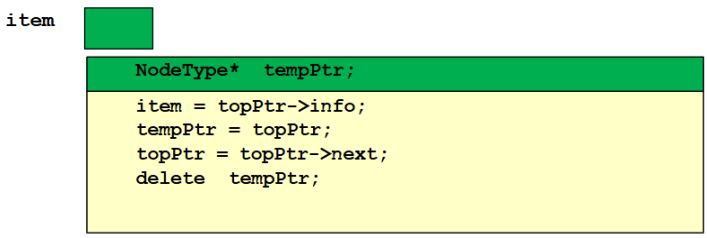

```
char      letter = 'V';
StackType charStack;
charStack.Push(letter);
charStack.Push('C');
charStack.Push('S');
if ( !charStack.IsEmpty( )) charStack.Pop
     ( );
charStack.Push('K');
while (!charStack.IsEmpty( ))
{ letter = charStack.Top();
  charStack.Pop(0)}
```

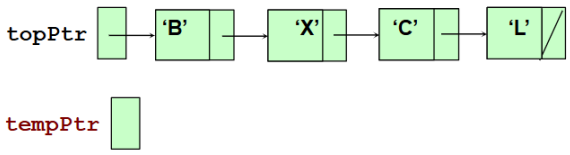
Stack Implementation: linked structure

- One advantage of an ADT is that the implementation can be changed without the program using it knowing about it.
- in-object array implementation: has a fixed max. size
- dynamically allocated array (as in lab2?): can be grown when needed, but lots of copy!
- Linked structure: **dynamically allocate the space for each element as it is pushed** onto stack.



Deleting top element from the stack

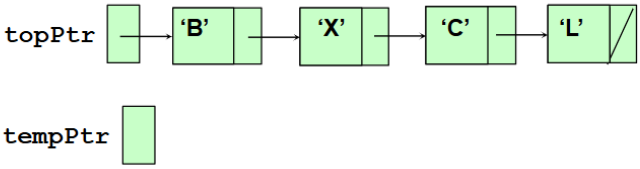




Deleting top element from the stack

item 'B'

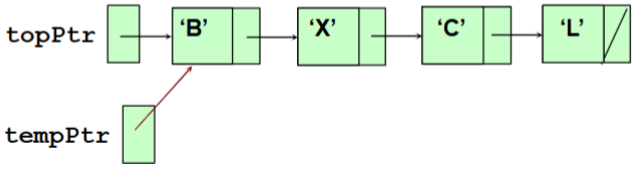
```
NodeType*  tempPtr;
item = topPtr->info;
tempPtr = topPtr;
topPtr = topPtr->next;
delete  tempPtr;
```

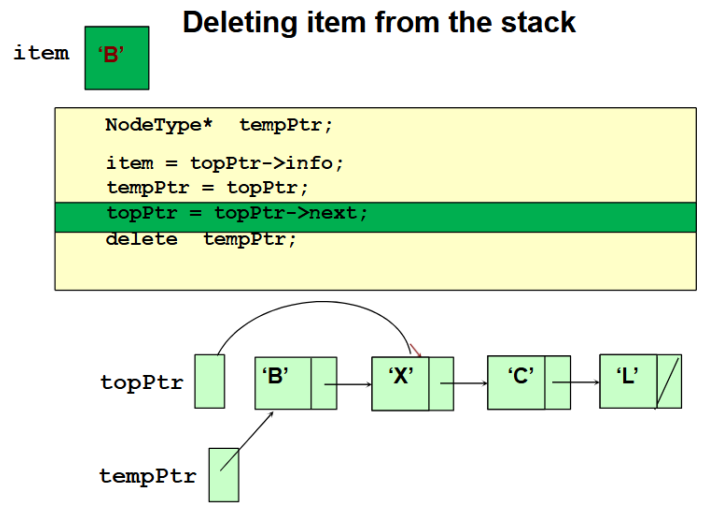


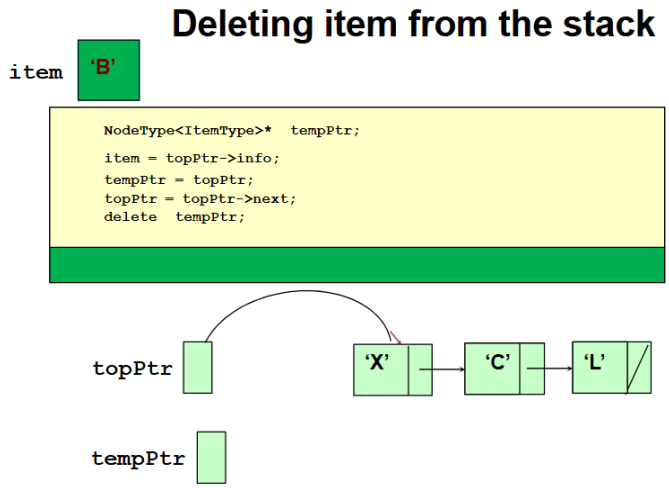
Deleting item from the stack

item 'B'

```
NodeType*  tempPtr;
item = topPtr->info;
tempPtr = topPtr;
topPtr = topPtr->next;
delete  tempPtr;
```







Array vs Linked Structure

- Drawback of array-based implementation:
    - at any point of time, array is either filled or not
    - memory is either not enough,
    - or wasted
- Linked Structure: allocate on demand
    - Drawback: need to store lots of addresses (in pointer field of node)

- if ItemType is small compared to pointer, then it's not memory efficient

Efficiency comparison

Time	Array-Based Implementation	Linked Implementation
Class constructor Class destructor	O(1) O(1)	O(1) O(n)
IsFull() IsEmpty()	O(1) O(1)	O(1) O(1)
Push() Pop()	O(1) O(1)	O(1) O(1)

C++ Standard Template Library

- a set of C++ class templates that provides common programming data structures and functions
- lists, stacks, queues, hash table, many more…
- all data structures/container are implemented as class template, which can be parameterized:
- vector<int>, vector<double> …
- stack<int>, stack<char>
- the type in <> is type parameter, specifying the type of items that the stack stores…

Sample code using STL stack

```
// CPP program to demonstrate working of STL stack
#include <iostream>
#include <stack>
using namespace std;

void showstack(stack <int> s)
{
    while (!s.empty())
    {
        cout << '\t' << s.top();
        s.pop();
    }
    cout << '\n';
}
```

Sample code using STL stack

```
int main ()
{
    stack <int> s;
    s.push(10);
    s.push(30);
    s.push(20);
    s.push(5);
    s.push(1);

    cout << "The stack is : ";
    showstack(s);

    cout << "\ns.size() : " << s.size();
    cout << "\ns.top() : " << s.top();


    cout << "\ns.pop() : ";
    s.pop();
    showstack(s);

    return 0;
}
```

Module 5 - Queue ADT

Module 5 - Queue ADT

Queue ADT

Queues

- Definition of a Queue
- Examples of Queues
- Design of a Queue Class
- Different Implementations of the Queue Class

Introduction to Queues

- A queue is a waiting line
- It's in daily life:
    - A line of persons waiting to check out at a supermarket
    - A line of persons waiting to purchase a ticket for a film
- A line of planes waiting to take off at an airport
- A line of vehicles at a toll booth

Definition of a Queue

- A queue is a data structure that models/enforces the **first--come first--serve order**, or equivalently **the first--in first--out** (FIFO) order.
- That is, the element that is inserted first into the queue will be the element that will deleted first, and the element that is inserted last is deleted last.
- A waiting line is a good real--life example of a queue. (In fact, the British word for "line" is "queue".)

Queues

Unlike stacks in which elements are popped and pushed only at the ends of the list

Collection of data elements:

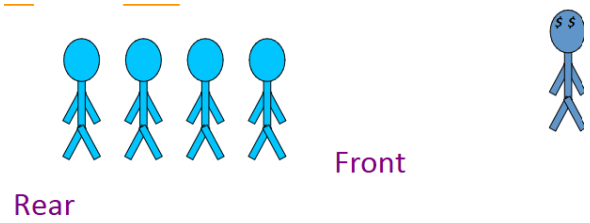
- items are removed from a queue at one end, called the FRONT of the queue;
- and elements are added at the other end, called the BACK

Introduction to Queues

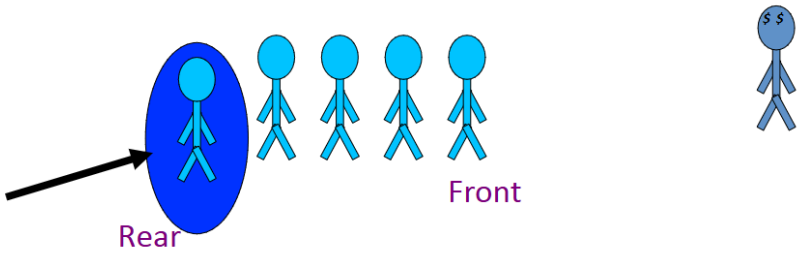
- Difference between Stack and Queues:
    - Stack exhibits last-in-first-out (LIFO)
    - Queue exhibits first-in-first-out (FIFO)

The Queue Operations

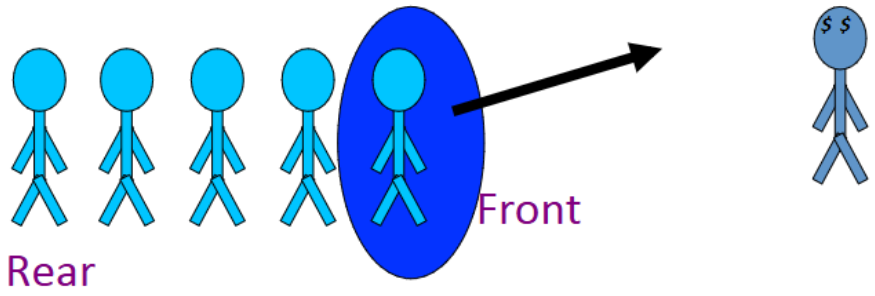
- A queue is like a line of people waiting for a bank teller. The queue has a **front** and a **rear**.



- New people must enter the queue at the rear. This is called an **enqueue** operation.



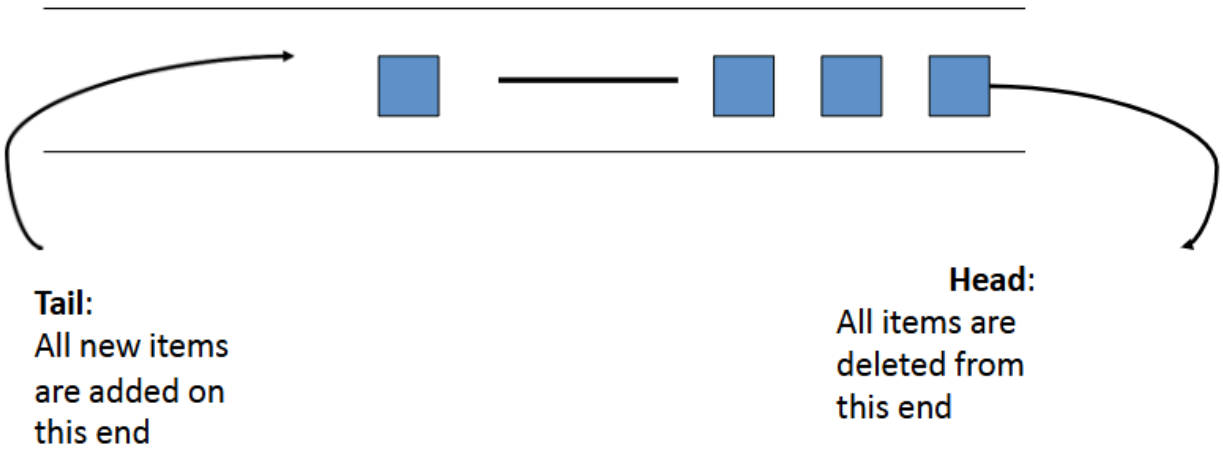
- When an item is taken from the queue, it always comes from the front. This is called a **dequeue** operation.



Queue Abstract Data Type

- Basic operations
    - Construct a queue
    - Check if empty
    - Enqueue (add element to back)
    - Front (retrieve value of element from front)
    - Dequeue (remove element from front)

A Graphic Model of a Queue



Operations on Queues

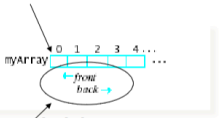
- Insert(item): (also called enqueue)
    - It adds a new item to the tail of the queue
- Remove( ): (also called delete or dequeue)
    - It deletes the head item of the queue, and returns to the caller. If the queue is already empty, this operation returns NULL
- getHead( ):
    - Returns the value in the head element of the queue
- getTail( ):
    - Returns the value in the tail element of the queue
- isEmpty( )
    - Returns true if the queue has no items
- size( )
    - Returns the number of items in the queue

Queue as a Class

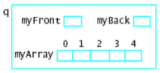
- Much like stacks and linked lists were designed and implemented as classes, a queue can be conveniently packaged as a class
- It seems natural to think of a queue as similar to a linked list, but with more basic operations, to enforce the FIFO order of insertion and deletion

Designing and Building a Queue Class Array-Based

• Consider an array in which to store a queue

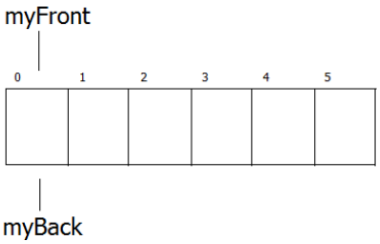


- Note additional variables needed
  - `myFront, myBack`
- Picture a queue object like this

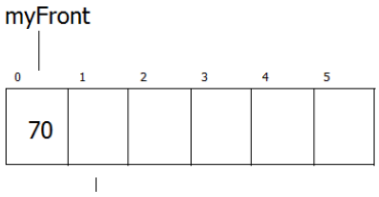


Queue Operation

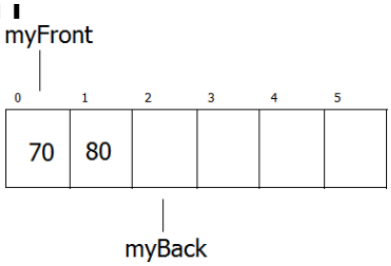
- Empty Queue



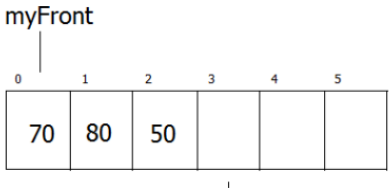
Enqueue(70)



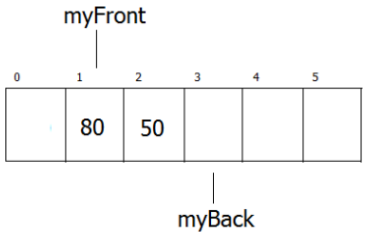
- Enqueue(80)



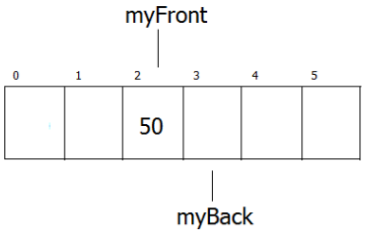
- Enqueue(50)



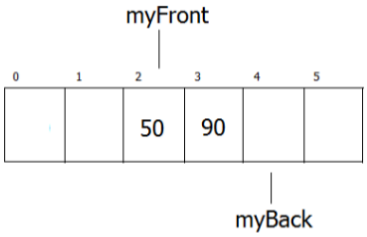
- Dequeue()



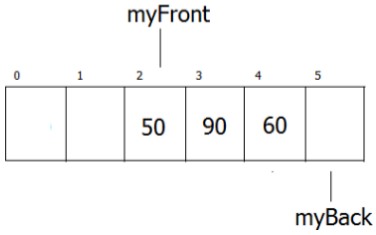
- Dequeue()



- Enqueue(90)



- Enqueue(60)

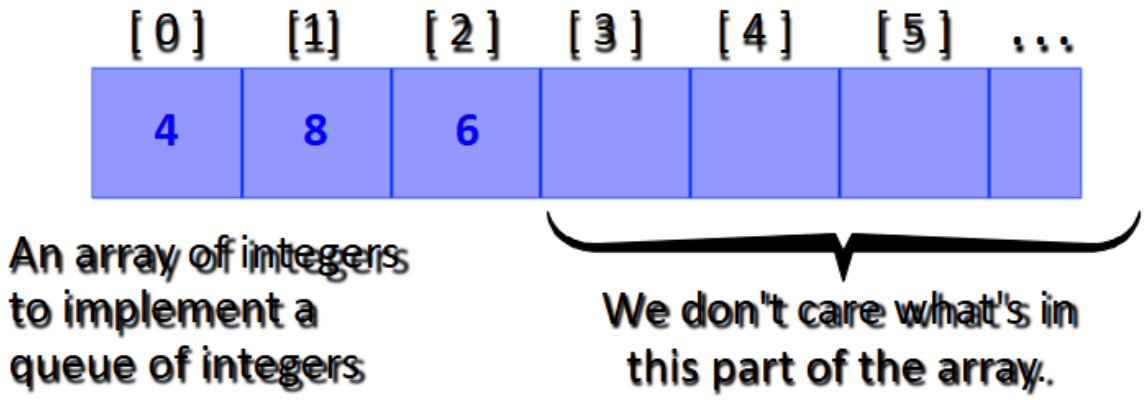


Circular Queue

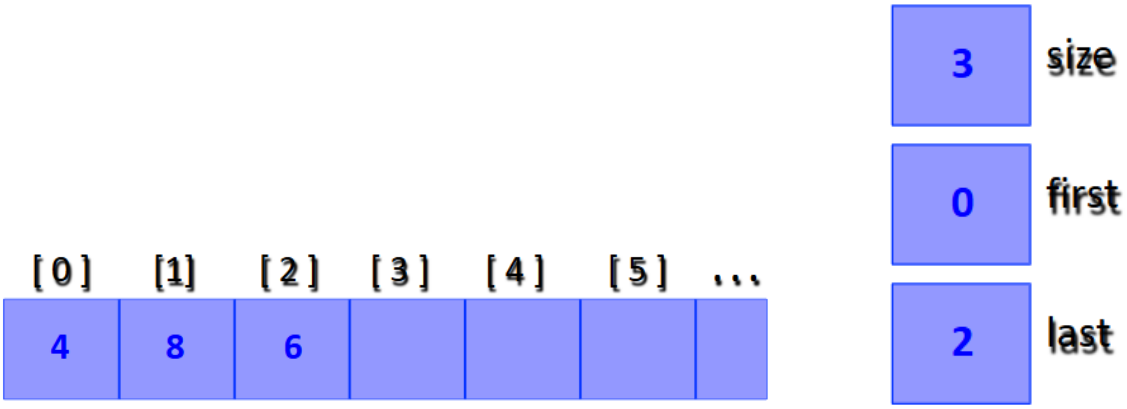
- Problems
  - We quickly "walk off the end" of the array
- Possible solutions
  - Shift array elements
  - Use a circular queue
  - Note that both empty and full queue gives *myBack == myFront*

Array Implementation

- A queue can be implemented with an array, as shown here. For example, this queue contains the integers 4 (at the front), 8 and 6 (at the rear).

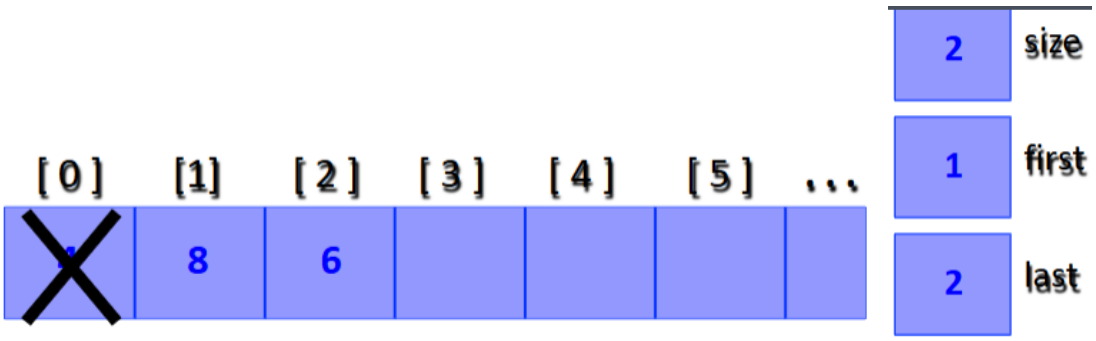


- The easiest implementation also keeps track of the number of items in the queue and the index of the first element (at the front of the queue), the last element (at the rear).



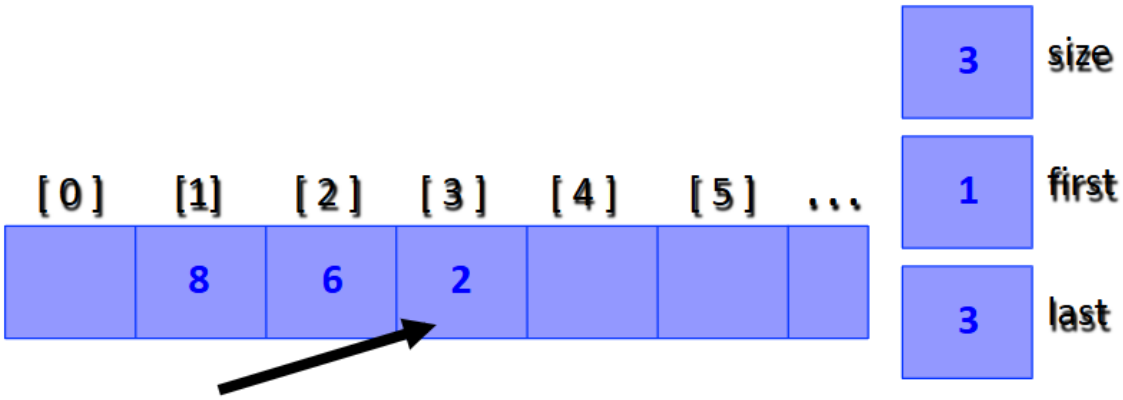
A Dequeue Operation

- When an element leaves the queue, size is decremented, and first changes, too.



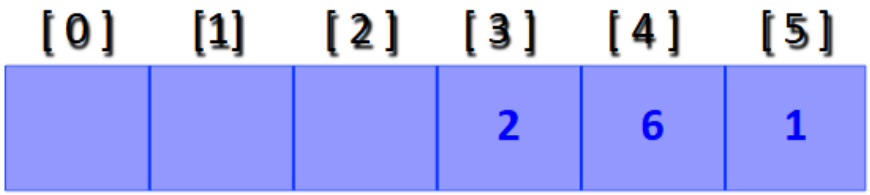
An Enqueue Operation

- When an element enters the queue, size is incremented, and last changes, too.



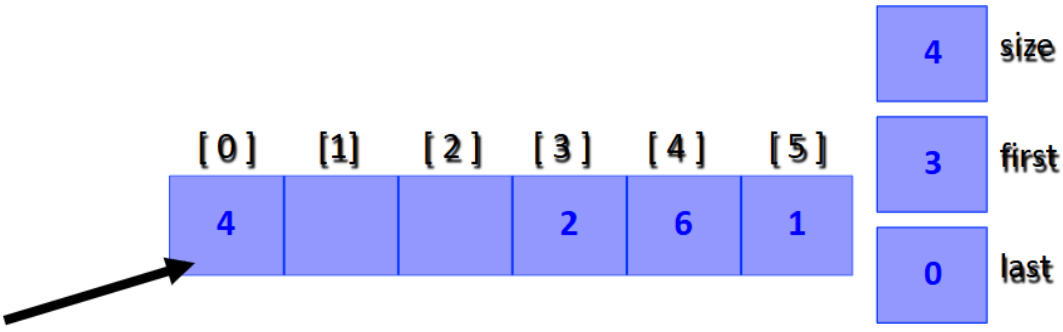
At the End of the Array

- There is special behavior at the end of the array. For example, suppose we want to add a new element to this queue, where the last index is [5]:



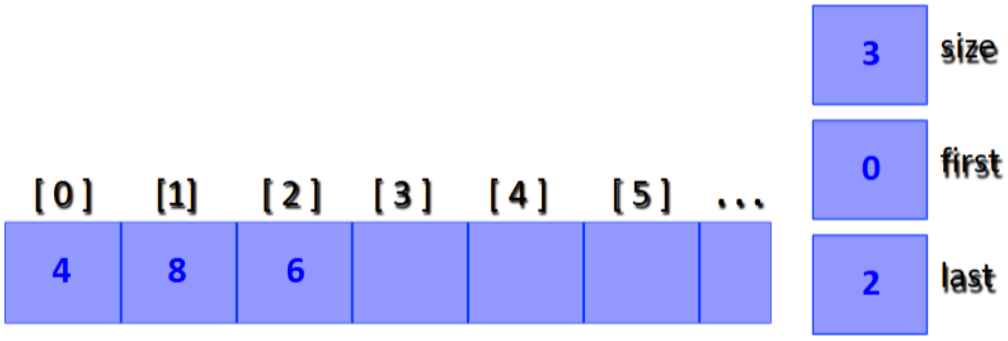


- The new element goes at the front of the array (if that spot isn't already used):



Array Implementation

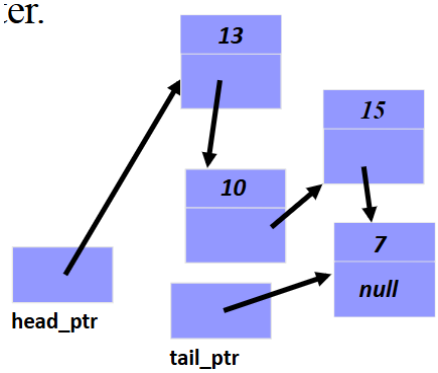
- Easy to implement
- But it has a limited capacity with a fixed array
- Or you must use a dynamic array for an unbounded capacity
- Special behavior is needed when the rear reaches the end of the array.



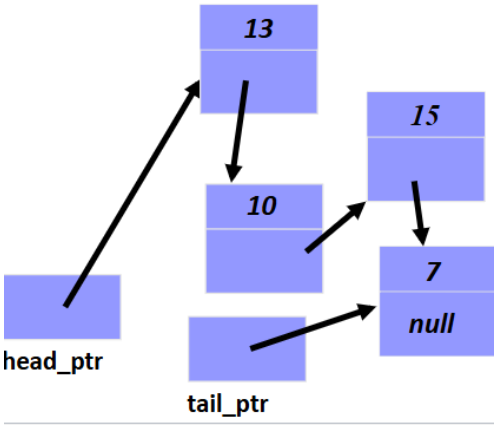
Linked List Implementation

- A queue can also be implemented with a linked list with both a head and a tail pointer.

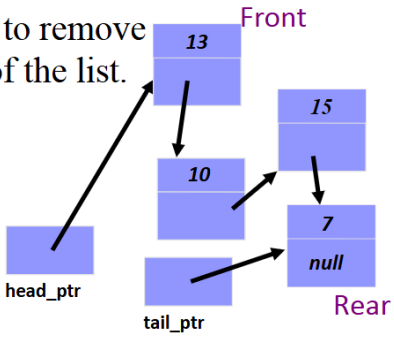
er.



- Which end do you think is the front of the queue? Why?



- The head_ptr points to the front of the list.
- Because it is harder to remove items from the tail of the list.



Abstract Data Type

- Abstract Data Type as a design tool
- Concerns only on the important concept or model
- No concern on implementation details.
- Stack & Queue is an example of ADT

- An array is not ADT.

What is the difference?

- Stack & Queue vs. Array
    - Arrays are data storage structures while stacks and queues are specialized DS and used as programmer's tools.
- Stack
    - a container that allows push and pop
- Queue
    - A container that allows enqueue and dequeue
- No concern on implementation details.
- In an array any item can be accessed, while in these data structures access is restricted.
- They are more abstract than arrays.